

Project Overview: Quantum-Assisted Collaborative Filtering (QACF)

My Role: I developed a hybrid quantum-classical pipeline to improve movie recommendations on the MovieLens 32M dataset, specifically targeting the "Sparsity Problem."

1. The Problem I Solved

In modern recommendation systems, the data is sparse: the average user has only rated a tiny fraction of the 84,000 available movies. Standard classical algorithms (like K-Means) often struggle to find meaningful patterns in this "empty" data. My work proves that Quantum Kernels can "see" deeper similarities between items by mapping them into a higher-dimensional Hilbert Space, leading to more robust recommendations when data is scarce.

2. My Step-by-Step Technical Workflow (1–30)

Phase 1: Industrial Data Engineering (Steps 1–5)

- What I did: I built a data pipeline capable of handling 32 million ratings. Since this is too large for standard RAM, I used chunked loading and Sparse CSR Matrices to keep the memory footprint under 8GB (avoiding a 140GB crash).
- The Bridge: I used Truncated SVD to compress 84,000 movie dimensions into 32 latent features. This allows the data to "fit" into current quantum simulators.

Phase 2: Establishing the Classical Baseline (Steps 6–15)

- What I did: I implemented a standard Item-Based Collaborative Filtering model using MiniBatch K-Means.
- Result: This gave me a benchmark for Hit Rate@10 and RMSE. I found that while classical models are fast, they are easily "confused" by items with very few ratings.

Phase 3: Building the Quantum Engine (Steps 16–19)

- What I did: I translated classical movie features into quantum states using Amplitude and Angle Encoding.
- The Innovation: I developed a Quantum Kernel in both Qiskit and PennyLane.

Instead of measuring "Euclidean distance," I used Quantum Fidelity (the overlap of wave functions) to determine if two movies are similar.

Phase 4: Hybrid Recommendation Loop (Steps 20–24)

- What I did: I replaced the classical similarity logic with my Quantum Kernel and generated a new "Top 10" recommendation list for users.
- Evaluation: I compared the two side-by-side. I observed that the Quantum model had higher Coverage, meaning it suggested a more diverse range of movies than the classical model.

Phase 5: Research Validation & Stress Testing (Steps 25–30)

- The Stress Test (Step 27): I artificially "starved" the model by dropping 80% of the training data. My Quantum Kernel retained higher accuracy than the classical model, proving its robustness.
- The Reality Check (Step 26): I added Shot Noise (1024 shots) to simulate a real, imperfect quantum computer. The model still performed well, proving it is "hardware-ready."
- Scientific Rigor (Step 30): I ran Bootstrap Significance Tests to ensure my results were statistically valid and not just due to a lucky sample.

3. My Research Significance (The "Why")

Why this is relevant to my internship company:

1. Beating the "Cold Start": My results show that we can provide better recommendations to new users or for niche, low-rated movies where classical math fails.
2. Hybrid Efficiency (Protocol D): I designed a hybrid strategy where we use fast Classical methods for popular "Blockbuster" movies and deep Quantum kernels for "Long-tail" niche movies. This balances speed and depth.
3. Future Proofing: By building this in Qiskit and PennyLane with noise simulation, I've created a roadmap for when the company eventually moves from simulators to real quantum hardware.

4. My Evaluation of the Project

- What went well: I proved that Quantum-Assisted clustering is more robust to data sparsity. I also successfully managed a massive 32M row dataset which was a major technical challenge.
- What I learned: I learned that in the real world, Ranking Accuracy (Hit Rate) is more important than Prediction Error (RMSE). A user doesn't care if we predicted 4.2 stars instead of 4.5; they care if the movie we recommended is actually something they want to watch.
- Next Steps: If I had more time, I would explore Quantum Neural Networks (QNNs) to further optimize the latent space after the SVD step.